

Sign Takeoff IQ — Full Rebuild Specification

Paste this entire document into Replit Agent as the build prompt

PRODUCT OVERVIEW

Build "Sign Takeoff IQ" — a multi-tenant SaaS application that extracts interior signage data from architectural construction document PDFs. Users upload plan PDFs, the system automatically identifies every room and assigns sign types using a deterministic rules engine, then presents results as interactive markers on the floor plan with a companion data table. Users can add, edit, and delete markers manually. Corrections feed back into a training database that improves accuracy over time per tenant.

The app has an existing dark-theme UI style that must be preserved. Screenshots of the current app are provided as reference for the visual design language.

TECH STACK

- **Frontend:** Next.js 14+ (App Router) with TypeScript, Tailwind CSS
- **Backend:** Python FastAPI (for PDF processing — Python has the best PDF/AI library ecosystem)
- **Database:** Supabase (PostgreSQL with Row Level Security for multi-tenant isolation)
- **Auth:** Supabase Auth (email + password, magic link, Google OAuth)
- **Storage:** Supabase Storage (for uploaded PDFs, rasterized pages, generated outputs)
- **LLM:** Claude API (Anthropic) via [anthropic](#) Python SDK — specifically Claude Sonnet for vision tasks. Only 2-3 targeted API calls per takeoff, not full-plan analysis.
- **PDF Processing:** [pdfplumber](#) (text + position extraction), [pdf2image/poppler](#) (rasterization), [reportlab](#) + [pypdf](#) (marked-up PDF generation)
- **Deployment:** Replit (monorepo — Next.js frontend + FastAPI backend)

DESIGN SYSTEM

Dark theme matching the existing app screenshots:

- Background: [#0d0d0f](#) (near-black)
- Surface/cards: [#1a1a1e](#) with subtle [#2a2a2e](#) borders
- Primary accent: [#f59e0b](#) (amber/gold — used for CTAs, active states, the logo accent)

- Text primary: #e5e5e5
- Text secondary: #9ca3af
- Success: #22c55e (green — DONE badges, high confidence)
- Warning: #f59e0b (amber — FLAG status, needs review)
- Danger: #ef4444 (red — EXCEPTION status, delete actions)
- Info: #3b82f6 (blue)
- Font: System sans-serif stack, all-caps for headings/labels matching the current style
- Border radius: 8px for cards, 4px for badges/pills
- The overall feel is professional, dark, data-dense — like a Bloomberg terminal for sign estimators

Sign type color coding (for markers and legend)

These colors are used for both floor plan markers and table badges:

- Room ID: #f59e0b (amber/gold) — most common, the "default" sign
- Room ID w/insert: #a855f7 (purple)
- Restroom: #14b8a6 (teal)
- Exit: #ef4444 (red)
- Max Occupancy: #f97316 (orange)
- Stair (Corridor): #64748b (slate)
- Stair (Landing): #94a3b8 (light slate)
- Evacuation Map: #22c55e (green)
- In Case of Fire: #ec4899 (pink)
- Office Directory: #3b82f6 (blue)
- Elevator signs: #22c55e (green)

DATABASE SCHEMA (Supabase/PostgreSQL)

Multi-tenant architecture

Every data table has a `tenant_id` column with Row Level Security (RLS) policies ensuring tenants can only see their own data. The `tenants` table stores org-level settings.

-- Tenants (organizations / sign companies)

```
CREATE TABLE tenants (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  name TEXT NOT NULL,
```

```
  slug TEXT UNIQUE NOT NULL,
```

```

plan TEXT DEFAULT 'free', -- free, pro, enterprise

settings JSONB DEFAULT '{}',

created_at TIMESTAMPTZ DEFAULT now()

);

-- Users (belong to a tenant)

CREATE TABLE users (

    id UUID PRIMARY KEY REFERENCES auth.users(id),

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    email TEXT NOT NULL,

    full_name TEXT,

    role TEXT DEFAULT 'member', -- super_admin, admin, member, viewer

    created_at TIMESTAMPTZ DEFAULT now()

);

-- Jobs (one per uploaded plan set)

CREATE TABLE jobs (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    name TEXT NOT NULL,

    status TEXT DEFAULT 'uploading', -- uploading, processing, completed, failed, archived

    location TEXT, -- e.g., "BOSTON, MA"

    jurisdiction TEXT, -- e.g., "Massachusetts (521 CMR MAAB)"

```

building_type TEXT, -- auto-detected or user-set: residential, commercial, school, hospital, hotel, etc.

file_count INTEGER DEFAULT 0,

total_signs INTEGER DEFAULT 0,

high_confidence INTEGER DEFAULT 0,

needs_review INTEGER DEFAULT 0,

ai_token_cost NUMERIC(10,4) DEFAULT 0,

metadata JSONB DEFAULT '{}', -- project name, architect, issue date, etc.

created_by UUID REFERENCES users(id),

created_at TIMESTAMPTZ DEFAULT now(),

updated_at TIMESTAMPTZ DEFAULT now()

);

-- Buildings within a job (for multi-building projects like NOVO 292+294 Baker)

CREATE TABLE buildings (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

tenant_id UUID REFERENCES tenants(id) NOT NULL,

name TEXT NOT NULL, -- e.g., "292 Baker Ave" or "Building A"

address TEXT,

building_type TEXT, -- can override job-level type for mixed-use

level_count INTEGER,

total_signs INTEGER DEFAULT 0,

```

    sort_order INTEGER DEFAULT 0,

    created_at TIMESTAMPTZ DEFAULT now()

);

-- Uploaded PDF files (per job)

CREATE TABLE job_files (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    filename TEXT NOT NULL,

    storage_path TEXT NOT NULL, -- Supabase storage path

    page_count INTEGER,

    file_size_bytes BIGINT,

    created_at TIMESTAMPTZ DEFAULT now()

);

-- Sheets identified from the drawing index

CREATE TABLE job_sheets (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    file_id UUID REFERENCES job_files(id),

    sheet_id TEXT NOT NULL, -- e.g., "A-101"

    sheet_title TEXT,

```

pdf_page INTEGER NOT NULL, -- 1-indexed

sheet_type TEXT, -- 'floor_plan', 'signage_schedule', 'egress', 'code_review', 'other'

rasterized_path TEXT, -- storage path to rasterized image

level TEXT, -- e.g., "LEVEL 1", null for non-floor-plan sheets

is_relevant BOOLEAN DEFAULT false

);

-- Plaque schedule (extracted from signage sheet)

CREATE TABLE plaque_schedules (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

tenant_id UUID REFERENCES tenants(id) NOT NULL,

type_id TEXT NOT NULL, -- "A", "B", "EXIT", etc.

name TEXT NOT NULL,

braille BOOLEAN DEFAULT false,

has_insert BOOLEAN DEFAULT false,

insert_size TEXT,

letter_height TEXT,

trigger_description TEXT,

maps_to_column TEXT, -- "Room ID", "Restroom", etc.

material_notes TEXT,

raw_json JSONB

);

-- Room inventory (all rooms found on plans)

CREATE TABLE rooms (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

tenant_id UUID REFERENCES tenants(id) NOT NULL,

building_id UUID REFERENCES buildings(id), -- for multi-building projects

sheet_id UUID REFERENCES job_sheets(id),

room_number TEXT NOT NULL,

room_name TEXT NOT NULL,

level TEXT NOT NULL,

-- Coordinates (normalized 0-1000, origin bottom-left)

coord_x INTEGER,

coord_y INTEGER,

bbox_x0 NUMERIC, bbox_y0 NUMERIC, bbox_x1 NUMERIC, bbox_y1 NUMERIC,

-- From occupant loads table

occupant_load INTEGER,

occupancy_group TEXT,

-- Derived flags

is_residential_unit BOOLEAN DEFAULT false, -- UNIT xxx rooms (R16)

is_restroom BOOLEAN DEFAULT false,

is_stair BOOLEAN DEFAULT false,

```

is_elevator BOOLEAN DEFAULT false,

is_vestibule BOOLEAN DEFAULT false,

is_corridor_or_hall BOOLEAN DEFAULT false,

is_vehicle_bay BOOLEAN DEFAULT false,

is_mep_unoccupied BOOLEAN DEFAULT false,

is_variable_use BOOLEAN DEFAULT false,

is_public_facing BOOLEAN DEFAULT false,

is_staff_only BOOLEAN DEFAULT false,

is_assembly BOOLEAN DEFAULT false,

-- Discovery metadata

source TEXT DEFAULT 'auto', -- 'auto' (extracted), 'manual' (user-added)

was_missed_by_human BOOLEAN DEFAULT false,

created_at TIMESTAMPTZ DEFAULT now()

);

-- Signs (the actual takeoff — one row per sign assigned to a room)

CREATE TABLE signs (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    building_id UUID REFERENCES buildings(id), -- for multi-building projects

    room_id UUID REFERENCES rooms(id) ON DELETE CASCADE,

```

-- Sign details

sign_type TEXT NOT NULL, -- "Room ID", "Restroom", "Exit", etc.

plaque_type_id TEXT, -- references plaque_schedules.type_id

qty INTEGER DEFAULT 1,

-- Marker position on the floor plan (normalized 0-1000)

marker_x INTEGER,

marker_y INTEGER,

sheet_id UUID REFERENCES job_sheets(id), -- which floor plan page

-- Rule traceability

rule_ref TEXT, -- "R1", "R2", etc.

color TEXT, -- hex color for the marker

-- Confidence and review

confidence NUMERIC(3,2) DEFAULT 0.0, -- 0.00 to 1.00

status TEXT DEFAULT 'auto', -- 'auto', 'confirmed', 'edited', 'flagged', 'exception', 'deleted'

source TEXT DEFAULT 'rule_engine', -- 'rule_engine', 'llm', 'manual', 'training'

-- Spec details (filled by LLM or manually)

dimensions TEXT,

mounting TEXT,

finish_color TEXT,

message TEXT,

-- Audit trail

created_by UUID REFERENCES users(id),

```

    edited_by UUID REFERENCES users(id),

    created_at TIMESTAMPTZ DEFAULT now(),

    updated_at TIMESTAMPTZ DEFAULT now()

);

-- Training corrections (feedback loop)

CREATE TABLE training_corrections (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    job_id UUID REFERENCES jobs(id),

    sign_id UUID REFERENCES signs(id),

    room_id UUID REFERENCES rooms(id),

    -- What changed

    correction_type TEXT NOT NULL, -- 'added', 'edited', 'deleted', 'reclassified'

    original_value JSONB, -- snapshot before correction

    corrected_value JSONB, -- what it was changed to

    -- Pattern for rule learning

    room_name_pattern TEXT, -- e.g., "CELL" → helps learn that cells get Room ID

    sign_type TEXT,

    rule_ref TEXT,

    reason TEXT, -- user's reason for correction

    -- Applied to future jobs?

```

```

is_active BOOLEAN DEFAULT true,

applied_count INTEGER DEFAULT 0, -- how many times this correction was auto-applied

created_by UUID REFERENCES users(id),

created_at TIMESTAMPTZ DEFAULT now()

);

-- Rule overrides (tenant-specific rule modifications learned from training)

CREATE TABLE rule_overrides (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    tenant_id UUID REFERENCES tenants(id) NOT NULL,

    rule_ref TEXT NOT NULL, -- "R1", "R5", etc.

    override_type TEXT NOT NULL, -- 'include', 'exclude', 'modify_qty', 'change_type'

    condition JSONB NOT NULL, -- e.g., {"room_name_contains": "CELL", "building_type":
"public_safety"}

    action JSONB NOT NULL, -- e.g., {"sign_type": "Room ID", "qty": 1}

    confidence NUMERIC(3,2) DEFAULT 0.5,

    source_corrections INTEGER DEFAULT 0, -- how many corrections generated this override

    is_active BOOLEAN DEFAULT true,

    created_at TIMESTAMPTZ DEFAULT now()

);

-- Validation results (per job)

CREATE TABLE validation_results (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

job_id UUID REFERENCES jobs(id) ON DELETE CASCADE NOT NULL,

tenant_id UUID REFERENCES tenants(id) NOT NULL,

check_name TEXT NOT NULL,

status TEXT NOT NULL, -- 'pass', 'fail', 'warning'

details TEXT,

created_at TIMESTAMPTZ DEFAULT now()

);

-- RLS policies (apply to ALL tables)

-- Example for jobs table (repeat pattern for all tables):

ALTER TABLE jobs ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Tenant isolation" ON jobs

    USING (tenant_id = (SELECT tenant_id FROM users WHERE id = auth.uid()));

```

BACKEND — FastAPI (Python)

Processing Pipeline

The pipeline runs asynchronously when a job is created. Use Supabase Edge Functions or a background worker (Python asyncio task) to process.

POST /api/jobs/{job_id}/process → kicks off the pipeline

Pipeline steps (in order):

1. intake() → extract PDF metadata, page count
2. filterSheets() → parse drawing index, identify relevant pages
3. rasterize() → render relevant pages to images, store in Supabase Storage
4. extractPlaque() → [CLAUDE API CALL #1] read signage schedule page

5. extractRooms() → [DETERMINISTIC] pdfplumber text+position extraction
6. extractLoads() → [CLAUDE API CALL #2] read occupant loads table
7. buildInventory() → merge rooms + loads, derive flags, check rule overrides
8. applyRules() → [DETERMINISTIC] R1-R15 rules engine
9. verify() → [DETERMINISTIC] 10 validation checks
10. saveResults() → write signs + rooms + validation to database

Claude API calls — only 2-3 per job

Call #1 — Plaque schedule extraction: Send 1 rasterized signage-schedule page to Claude Sonnet. System prompt: "Extract plaque types from this architectural signage schedule. Return JSON only." Cost: ~2,000 input tokens + ~500 output tokens.

Call #2 — Occupant loads table: Send 1 rasterized egress page to Claude Sonnet. System prompt: "Extract room occupant loads and occupancy groups from this table. Return JSON only." Cost: ~2,000 input tokens + ~1,000 output tokens.

Call #3 (optional) — Ambiguity resolution: Only fires if rooms have unclear classification. Batch all ambiguities into one call. Cost: ~3,000 tokens when needed.

Total cost per takeoff: ~\$0.01-0.03 (vs ~\$4-6 in the current approach).

Rules Engine (Python — deterministic, no LLM)

The rules engine uses a **trait-based system** where each building type maps to a set of trait values that control how rules R1-R15 behave. This avoids hardcoding separate rule sets per building type.

```
# building_traits.py
```

```
from dataclasses import dataclass
```

```
from enum import Enum
```

```
class MepPolicy(Enum):
```

```
    ALL = "all"          # Every MEP room gets Room ID
```

```
OCCUPIED_ONLY = "occupied" # Only MEP rooms with occupant_load > 0
```

```
MINIMAL = "minimal" # MEP rooms excluded unless scoped
```

```
class ElevatorMode(Enum):
```

```
    PER_BUILDING = "per_building" # 1x "In Case of Fire" per elevator total
```

```
    PER_LEVEL = "per_level" # 1x Elevator ID per elevator per level
```

```
class StairMode(Enum):
```

```
    SEPARATE = "separate" # Corridor and Landing as separate columns
```

```
    COMBINED = "combined" # Single column, value = corridor + landing
```

```
@dataclass
```

```
class BuildingTraits:
```

```
    has_unit_numbers: bool = False
```

```
    mep_policy: MepPolicy = MepPolicy.OCCUPIED_ONLY
```

```
    has_assembly_rules: bool = True
```

```
    has_directory: bool = False
```

```
    elevator_mode: ElevatorMode = ElevatorMode.PER_BUILDING
```

```
    stair_mode: StairMode = StairMode.SEPARATE
```

```
# Building type → trait mapping
```

```
BUILDING_TRAITS = {
```

```
    "residential": BuildingTraits(True, MepPolicy.ALL, False, False,  
    ElevatorMode.PER_LEVEL, StairMode.COMBINED),
```

```
    "commercial": BuildingTraits(False, MepPolicy.OCCUPIED_ONLY, True, True,  
    ElevatorMode.PER_BUILDING, StairMode.SEPARATE),
```

```

    "school":      BuildingTraits(False, MepPolicy.MINIMAL,      True, False,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "hospital":    BuildingTraits(False, MepPolicy.MINIMAL,      True, False,
ElevatorMode.PER_LEVEL,   StairMode.SEPARATE),

    "hotel":       BuildingTraits(True, MepPolicy.ALL,           True, False,
ElevatorMode.PER_LEVEL,   StairMode.COMBINED),

    "retail":      BuildingTraits(False, MepPolicy.ALL,          True, False,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "warehouse":   BuildingTraits(False, MepPolicy.ALL,          False, False,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "lab":         BuildingTraits(False, MepPolicy.OCCUPIED_ONLY, False, True,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "bank":        BuildingTraits(False, MepPolicy.OCCUPIED_ONLY, False, True,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "government":  BuildingTraits(False, MepPolicy.OCCUPIED_ONLY, True, True,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "church":      BuildingTraits(False, MepPolicy.OCCUPIED_ONLY, True, False,
ElevatorMode.PER_BUILDING, StairMode.SEPARATE),

    "senior_living": BuildingTraits(True, MepPolicy.ALL,          True, False,
ElevatorMode.PER_LEVEL,   StairMode.COMBINED),

}

```

```

def detect_building_type(rooms: list[RoomInventory], metadata: dict) -> str:

```

```

    """Auto-detect building type from room names and project metadata."""

```

```

    all_names = " ".join(r.room_name.upper() for r in rooms)

```

```

    # Check metadata first (explicit project type)

```

```

    if metadata.get("building_type"):

```

```

return metadata["building_type"]

# Count UNIT rooms

unit_count = sum(1 for r in rooms if r.room_name.upper().startswith("UNIT"))

if unit_count > len(rooms) * 0.3:

    return "residential"

# Keyword detection

if any(kw in all_names for kw in ["PATIENT", "NURSE STATION", "EXAM", "ICU", "OR"]):

    return "hospital"

if any(kw in all_names for kw in ["CLASSROOM", "GYMNASIUM", "CAFETERIA",
"PRINCIPAL"]):

    return "school"

if any(kw in all_names for kw in ["GUEST ROOM", "FRONT DESK", "HOUSEKEEPING"]):

    return "hotel"

if any(kw in all_names for kw in ["NAVE", "SANCTUARY", "ALTAR", "PARISH"]):

    return "church"

if any(kw in all_names for kw in ["CLEAN ROOM", "FUME HOOD", "BSL"]):

    return "lab"

if any(kw in all_names for kw in ["VAULT", "TELLER", "SAFE DEPOSIT"]):

    return "bank"

if any(kw in all_names for kw in ["LOADING DOCK", "SHIPPING", "ASSEMBLY LINE"]):

```

```
return "warehouse"
```

```
if any(kw in all_names for kw in ["MERCHANDISE", "FITTING ROOM", "POS", "STOCK ROOM"]):
```

```
return "retail"
```

```
# IBC occupancy group
```

```
occ = metadata.get("occupancy_group", "")
```

```
if occ.startswith("R"): return "residential"
```

```
if occ.startswith("E"): return "school"
```

```
if occ.startswith("I-2"): return "hospital"
```

```
if occ.startswith("A"): return "church" # or assembly venue
```

```
if occ.startswith("S") or occ.startswith("F"): return "warehouse"
```

```
return "commercial" # default
```

```
# rules_engine.py
```

```
def apply_rules(room: RoomInventory, traits: BuildingTraits,
```

```
    plaque_schedule: list, rule_overrides: list) -> list[SignAssignment]:
```

```
    """Apply R1-R17 signage rules to a single room, using building traits."""
```

```
signs = []
```

```
# Check tenant overrides first
```

```
for override in rule_overrides:
```

```
    if matches_condition(room, override.condition):
```

```
        signs.append(apply_override(room, override))
```

```
    if override.override_type == 'exclude':
```

```
        return signs
```

```
# R16: Unit/Room Number (residential, hotel, senior living)
```

```
if traits.has_unit_numbers and room.is_residential_unit:
```

```
    signs.append(SignAssignment(sign_type="Unit #", qty=1, rule_ref="R16",  
                                color="#f59e0b"))
```

```
# R1: Room/Service ID (skip units — they get R16 instead)
```

```
elif not room.is_residential_unit:
```

```
    should_get_room_id = True
```

```
# Check exclusions
```

```
if room.is_corridor_or_hall: should_get_room_id = False # R4
```

```
if room.is_vehicle_bay: should_get_room_id = False # R5
```

```
if room.is_stair: should_get_room_id = False
```

```
if room.is_elevator: should_get_room_id = False
```

R6: MEP exclusion (trait-dependent)

if room.is_mep_unoccupied:

if traits.mep_policy == MepPolicy.ALL:

should_get_room_id = True # residential/hotel: MEP gets ID

elif traits.mep_policy == MepPolicy.OCCUPIED_ONLY:

should_get_room_id = False # commercial: MEP excluded

elif traits.mep_policy == MepPolicy.MINIMAL:

should_get_room_id = False # school/hospital: MEP excluded

if should_get_room_id:

R2: Variable-use → insert (only if assembly rules apply)

if room.is_variable_use and traits.has_assembly_rules:

qty = 2 if room.is_assembly else 1

signs.append(SignAssignment(sign_type="Room ID w/insert",
 qty=qty, rule_ref="R2", color="#a855f7"))

else:

signs.append(SignAssignment(sign_type="Room ID",
 qty=1, rule_ref="R1", color="#f59e0b"))

R3: Multi-entry (3+ doors) increases Room ID qty

if room.door_count and room.door_count >= 3:

```
for s in signs:
```

```
    if s.sign_type in ("Room ID", "Room ID w/insert"):
```

```
        s.qty = room.door_count
```

```
        s.rule_ref = "R3"
```

```
# R8: Restroom
```

```
if room.is_restroom:
```

```
    variant = select_restroom_variant(room)
```

```
    signs.append(SignAssignment(sign_type="Restroom", qty=1, rule_ref="R8",  
                                color="#14b8a6", plaque_type_id=variant))
```

```
# R9: Exit
```

```
if room.is_vestibule:
```

```
    signs.append(SignAssignment(sign_type="Exit", qty=1, rule_ref="R9",  
                                color="#ef4444"))
```

```
if room.is_public_facing and "LOBBY" in room.room_name:
```

```
    signs.append(SignAssignment(sign_type="Exit", qty=1, rule_ref="R9",  
                                color="#ef4444"))
```

```
if traits.has_assembly_rules and room.is_assembly and (room.occupant_load or 0) >= 50:
```

```
    signs.append(SignAssignment(sign_type="Exit", qty=2, rule_ref="R9",  
                                color="#ef4444"))
```

```
# R10: Capacity (only if assembly rules apply)
```

```
if traits.has_assembly_rules and room.is_assembly:
```

```
    # For hotels/senior living: only on conference/assembly spaces, not guest rooms
```

```
    if not room.is_residential_unit:
```

```
        signs.append(SignAssignment(sign_type="Max Occupancy", qty=1, rule_ref="R10",
                                     color="#f97316"))
```

```
# R14: Office Directory (only if directory trait is true)
```

```
if traits.has_directory and room.is_corridor_or_hall and room.is_public_facing:
```

```
    signs.append(SignAssignment(sign_type="Office Directory", qty=1, rule_ref="R14",
                                 color="#3b82f6"))
```

```
# Assign coordinates (offset from room label)
```

```
for sign in signs:
```

```
    sign.marker_x = room.coord_x + 30
```

```
    sign.marker_y = room.coord_y
```

```
    sign.confidence = compute_confidence(sign, room, rule_overrides)
```

```
return signs
```

```
def apply_stair_rules(stairs: list, levels: list, traits: BuildingTraits) -> list:
```

```
    """R11: Stair signs — depends on stair_mode trait."""
```

```

signs = []

for stair in stairs:

    for level in levels:

        if traits.stair_mode == StairMode.SEPARATE:

            signs.append(SignAssignment(sign_type="Stair (Corridor)",

                                        qty=stair.corridor_entries, rule_ref="R11", level=level))

            signs.append(SignAssignment(sign_type="Stair (Landing)",

                                        qty=1, rule_ref="R11", level=level))

        else: # COMBINED

            qty = stair.corridor_entries + 1 # entries + 1 landing

            signs.append(SignAssignment(sign_type="Stairs (C+L)",

                                        qty=qty, rule_ref="R11", level=level))

    return signs

```

```

def apply_elevator_rules(elevators: list, levels: list, traits: BuildingTraits) -> list:

```

```

    """R12: Elevator signs — depends on elevator_mode trait."""

```

```

    signs = []

    if traits.elevator_mode == ElevatorMode.PER_BUILDING:

        for elev in elevators:

            signs.append(SignAssignment(sign_type="In Case of Fire",

                                        qty=1, rule_ref="R12"))

    else: # PER_LEVEL

```

```
for elev in elevators:
```

```
    for level in levels:
```

```
        signs.append(SignAssignment(sign_type="Elevator",
```

```
                                qty=1, rule_ref="R12", level=level))
```

```
    return signs
```

```
def apply_evac_map_rules(elevators: list, levels: list,
```

```
                        occupied_levels: list, traits: BuildingTraits) -> list:
```

```
    """R13: Evacuation maps — depends on elevator_mode trait."""
```

```
    signs = []
```

```
    if traits.elevator_mode == ElevatorMode.PER_LEVEL:
```

```
        # Residential/hotel: 1 per elevator per occupied floor (not garages)
```

```
        for elev in elevators:
```

```
            for level in occupied_levels:
```

```
                signs.append(SignAssignment(sign_type="Evacuation Map",
```

```
                                qty=1, rule_ref="R13", level=level))
```

```
    else:
```

```
        # Commercial: 1 at each elevator lobby + main entries
```

```
        for elev in elevators:
```

```
            signs.append(SignAssignment(sign_type="Evacuation Map",
```

```
                                qty=1, rule_ref="R13"))
```

```
        # Plus main public entries (usually 1-2 per building)
```

```
return signs
```

Training Feedback System

When a user adds, edits, or deletes a marker:

```
# training.py
```

```
async def record_correction(tenant_id, sign_id, correction_type,
```

```
                           original, corrected, user_id, reason=None):
```

```
    """Record a user correction and potentially create a rule override."""
```

```
# 1. Save the correction
```

```
correction = await db.training_corrections.insert({
```

```
    'tenant_id': tenant_id,
```

```
    'sign_id': sign_id,
```

```
    'correction_type': correction_type,
```

```
    'original_value': original,
```

```
    'corrected_value': corrected,
```

```
    'room_name_pattern': extract_pattern(original.get('room_name', "")),
```

```
    'sign_type': corrected.get('sign_type'),
```

```
    'reason': reason,
```

```
    'created_by': user_id
```

```
})
```

2. Check if this pattern has been corrected N+ times → create rule override

```
similar = await db.training_corrections.count(

    tenant_id=tenant_id,

    room_name_pattern=correction.room_name_pattern,

    correction_type=correction_type,

    is_active=True

)
```

OVERRIDE_THRESHOLD = 3 # after 3 similar corrections, create an override

if similar >= OVERRIDE_THRESHOLD:

```
    await create_or_update_override(tenant_id, correction)
```

async def create_or_update_override(tenant_id, correction):

"""Create a tenant-specific rule override from recurring corrections."""

```
existing = await db.rule_overrides.find(

    tenant_id=tenant_id,

    condition__room_name_contains=correction.room_name_pattern

)
```

if existing:

```
    existing.source_corrections += 1
```

```
    existing.confidence = min(0.95, existing.confidence + 0.1)
```

```
await existing.save()
```

```
else:
```

```
await db.rule_overrides.insert({  
  
    'tenant_id': tenant_id,  
  
    'rule_ref': correction.rule_ref or 'R1',  
  
    'override_type': correction.correction_type,  
  
    'condition': {'room_name_contains': correction.room_name_pattern},  
  
    'action': correction.corrected_value,  
  
    'confidence': 0.5,  
  
    'source_corrections': 1  
  
})
```

Room Extraction (deterministic — pdfplumber)

```
# room_extraction.py
```

```
import pdfplumber
```

```
import re
```

```
SYNONYMS = {  
  
    'VEST': ['VEST', 'VESTIBULE'],  
  
    'EMERG': ['EMERG', 'EMERGENCY'],  
  
    'INTERROG': ['INTERROG', 'INTERROGATION'],  
  
    'CORR': ['CORR', 'CORRIDOR'],  
  
}
```

```

def tokenize(s: str) -> set[str]:

    return {t for t in re.split(r'^A-Z0-9+', s.upper()) if len(t) >= 2}

def expand(tokens: set[str]) -> set[str]:

    out = set(tokens)

    for t in tokens:

        if t in SYNONYMS:

            out |= set(SYNONYMS[t])

    return out

def extract_rooms(pdf_path: str, page_num: int,

                  plan_bounds: dict = None) -> list[dict]:

    """Extract all room labels with coordinates from a floor plan page.

    Returns list of {room_number, room_name, center_x, center_y, bbox}."""

    with pdfplumber.open(pdf_path) as pdf:

        page = pdf.pages[page_num - 1] # 0-indexed

        words = page.extract_words()

        pdf_w, pdf_h = page.width, page.height

        # Default plan bounds (exclude right 20% for legend/notes)

        if not plan_bounds:

            plan_bounds = {'x_max': pdf_w * 0.82}

```

```
rooms = []
```

```
for w in words:
```

```
    text = w['text'].strip()
```

```
    m = re.match(r'^([1-9]\d{2}[A-Z]?)$', text)
```

```
    if not m:
```

```
        continue
```

```
    num = m.group(1)
```

```
    cx = (w['x0'] + w['x1']) / 2
```

```
    cy = (w['top'] + w['bottom']) / 2
```

```
    if cx > plan_bounds.get('x_max', pdf_w):
```

```
        continue
```

```
# Find CAPS text directly above
```

```
matched_words = []
```

```
for w2 in words:
```

```
    if w2 is w:
```

```
        continue
```

```
    cx2 = (w2['x0'] + w2['x1']) / 2
```

```
    cy2 = (w2['top'] + w2['bottom']) / 2
```

```
dy = cy - cy2
```

```
dx = abs(cx - cx2)
```

```
if 0 < dy < 45 and dx < 100:
```

```
    t2 = w2['text'].strip()
```

```
    if t2 and t2.isupper() and len(t2) >= 2:
```

```
        matched_words.append(w2)
```

```
if not matched_words:
```

```
    continue
```

```
# Build room name from matched words
```

```
name_parts = sorted(matched_words, key=lambda w: w['x0'])
```

```
room_name = ' '.join(w['text'].strip() for w in name_parts)
```

```
# Bounding box
```

```
all_boxes = [(w2['x0'], w2['top'], w2['x1'], w2['bottom'])
```

```
              for w2 in matched_words]
```

```
all_boxes.append((w['x0'], w['top'], w['x1'], w['bottom']))
```

```
bbox = {
```

```
    'x0': min(b[0] for b in all_boxes),
```

```
    'y0': min(b[1] for b in all_boxes),
```

```

        'x1': max(b[2] for b in all_boxes),

        'y1': max(b[3] for b in all_boxes)

    }

    center = {

        'x': (bbox['x0'] + bbox['x1']) / 2,

        'y': (bbox['y0'] + bbox['y1']) / 2

    }

    # Normalize to 0-1000 grid (bottom-left origin)

    norm_x = int((center['x'] / pdf_w) * 1000)

    norm_y = int((1 - center['y'] / pdf_h) * 1000) # flip Y

    rooms.append({

        'room_number': num,

        'room_name': room_name,

        'coord_x': norm_x,

        'coord_y': norm_y,

        'bbox': bbox,

        'pdf_page': page_num

    })

```

```
return rooms
```

Marked-up PDF Generation

```
# marked_pdf.py
```

```
from pypdf import PdfReader, PdfWriter
```

```
from reportlab.pdfgen import canvas
```

```
from reportlab.lib.colors import HexColor
```

```
import io
```

```
SIGN_COLORS = {
```

```
    'Room ID': '#f59e0b',
```

```
    'Room ID w/insert': '#a855f7',
```

```
    'Restroom': '#14b8a6',
```

```
    'Exit': '#ef4444',
```

```
    'Max Occupancy': '#f97316',
```

```
    'Stair (Corridor)': '#64748b',
```

```
    'Stair (Landing)': '#94a3b8',
```

```
    'Evacuation Map': '#22c55e',
```

```
    'In case of fire': '#ec4899',
```

```
    'Office Directory': '#3b82f6',
```

```
}
```

```
def generate_marked_pdf(original_pdf_path, signs_by_page, output_path):
```

```
    """Overlay sign markers onto floor plan pages."""
```

```
    reader = PdfReader(original_pdf_path)
```

```
writer = PdfWriter()
```

```
for page_num, page_signs in signs_by_page.items():
```

```
    page = reader.pages[page_num - 1]
```

```
    pdf_w = float(page.mediabox.width)
```

```
    pdf_h = float(page.mediabox.height)
```

```
    # Create overlay
```

```
    buf = io.BytesIO()
```

```
    c = canvas.Canvas(buf, pagesize=(pdf_w, pdf_h))
```

```
    for sign in page_signs:
```

```
        # Convert from normalized (0-1000) back to PDF coordinates
```

```
        px = (sign['marker_x'] / 1000) * pdf_w
```

```
        py = (sign['marker_y'] / 1000) * pdf_h # already bottom-left
```

```
        # Draw marker circle
```

```
        color = HexColor(sign.get('color', '#f59e0b'))
```

```
        c.setFillColor(color)
```

```
        c.setStrokeColor(HexColor('#000000'))
```

```
        c.setLineWidth(0.8)
```

```
c.circle(px, py, 8, stroke=1, fill=1)

# Room number label

c.setFillColor(HexColor('#ffffff'))

c.setFont("Helvetica-Bold", 6)

tw = c.stringWidth(sign['room_number'], "Helvetica-Bold", 6)

c.drawString(px - tw/2, py - 2, sign['room_number'])


# Add legend

draw_legend(c, page_signs, pdf_w, pdf_h)


c.save()

overlay = PdfReader(io.BytesIO(buf.getvalue())).pages[0]

page.merge_page(overlay)

writer.add_page(page)


with open(output_path, 'wb') as f:

    writer.write(f)
```

FRONTEND — Next.js Pages

Page structure

app/

(auth)/

login/page.tsx

register/page.tsx

(app)/

layout.tsx -- sidebar + top bar (dark theme)

dashboard/page.tsx -- All Takeoff Jobs list

upload/page.tsx -- New Extraction Job (drag & drop)

jobs/[id]/

layout.tsx -- job header + tab navigation

page.tsx -- redirects to sign-table

sign-table/page.tsx -- Sign Table view (editable)

floor-plans/page.tsx -- Floor Plans view with markers

sign-type-summary/page.tsx

sheets-analysis/page.tsx

sign-pages/page.tsx

sign-specs/page.tsx

coordinates/page.tsx

ai-scans/page.tsx

compliance/page.tsx

plaque-schedule/page.tsx

occupant-loads/page.tsx

timeline/page.tsx

training/page.tsx -- Training Import & corrections review

admin/page.tsx -- Admin Panel (tenant settings, users)

Floor Plan Viewer (the most complex component)

The floor plan viewer must support:

1. **Pan and zoom** — mouse wheel zoom, click-drag pan (use `react-zoom-pan-pinch` or similar)
2. **Markers rendered as HTML overlays** on top of the plan image
3. **Click a marker** → opens an edit popover with sign details
4. **Click empty space** → opens "Add marker" popover to manually place a sign
5. **Right-click marker** → delete option
6. **Legend overlay** — color-coded sign types with counts, toggleable visibility
7. **Page navigation** — "Floor plan 1/7 (pg 2) < >" for multi-page plans
8. **Export button** → generates marked-up PDF with current marker positions

// Simplified FloorPlanViewer component structure

```
function FloorPlanViewer({ jobId, sheetId }: Props) {

  const { signs, rooms, sheet } = useJobData(jobId, sheetId);

  const [selectedSign, setSelectedSign] = useState<Sign | null>(null);

  const [addingMarker, setAddingMarker] = useState(false);

  return (

    <div className="relative w-full h-full bg-[#0d0d0f]">

      {/* Zoom/pan wrapper */}

      <TransformWrapper>

        <TransformComponent>

          {/* Plan image */}
```

```
<img src={sheet.rasterizedUrl} className="max-w-none" />
```

```
{/* Marker overlay — positioned absolutely using normalized coords */}
```

```
{signs.map(sign => (
```

```
  <SignMarker
```

```
    key={sign.id}
```

```
    sign={sign}
```

```
    planWidth={sheet.imageWidth}
```

```
    planHeight={sheet.imageHeight}
```

```
    onClick={() => setSelectedSign(sign)}
```

```
    onRightClick={() => handleDelete(sign)}
```

```
  />
```

```
))}
```

```
</TransformComponent>
```

```
</TransformWrapper>
```

```
{/* Legend overlay (top-left) */}
```

```
<Legend signs={signs} />
```

```
{/* Marker count badge (bottom-right) */}
```

```
<div className="absolute bottom-4 right-4 bg-teal-600 px-3 py-1 rounded">
```

```
  {signs.length} MARKERS
```

```
</div>
```

```
  { /* Edit popover */
```

```
  {selectedSign && (
```

```
    <SignEditPopover
```

```
      sign={selectedSign}
```

```
      onSave={handleSave}
```

```
      onClose={() => setSelectedSign(null)}
```

```
      onDelete={() => handleDelete(selectedSign)}
```

```
    />
```

```
  )}
```

```
</div>
```

```
);
```

```
}
```

```
// Individual marker component
```

```
function SignMarker({ sign, planWidth, planHeight, onClick, onRightClick }) {
```

```
  const left = `${(sign.marker_x / 1000) * 100}%`;
```

```
  const bottom = `${(sign.marker_y / 1000) * 100}%`;
```

```
  return (
```

```
    <button
```

```

        className="absolute w-6 h-6 rounded-full border-2 border-black/50

        transform -translate-x-1/2 translate-y-1/2

        hover:scale-125 transition-transform cursor-pointer"

        style={{ left, bottom, backgroundColor: sign.color }}

        onClick={onClick}

        onContextMenu={(e) => { e.preventDefault(); onRightClick(); }}

    >

    <span className="text-[8px] font-bold text-white">

        {sign.room_number?.slice(-2)}

    </span>

</button>

);

}

```

Sign Table View (editable)

Match the existing table style from the screenshots:

- Columns: Sign ID, Location, Sign Type, Qty, Dimensions, Mounting, Finish/Color, Message, Confidence, Source, Status, Actions
- Confidence shown as percentage badge (green ≥80%, amber 50-79%, red <50%)
- Source badge: TEXT (blue), RULE (green), MANUAL (purple), LLM (amber)
- Status badge: auto (gray), confirmed (green), edited (blue), flagged (amber), exception (red)
- EDIT button on each row → inline editing
- Exception filter toggle at top ("EXCEPTIONS (222)")
- Sorting on all columns

Manual Marker Add/Edit/Delete

When a user edits a marker, the system must:

1. Update the `signs` table with new values
2. Create a `training_corrections` record with the before/after snapshot
3. Check if this correction pattern has occurred N+ times → auto-create a `rule_overrides` entry
4. Recompute the job's `total_signs`, `high_confidence`, `needs_review` counts

// API route: PATCH /api/jobs/{jobId}/signs/{signId}

```
async function updateSign(jobId: string, signId: string, updates: Partial<Sign>) {
```

```
  const original = await getSign(signId);
```

```
  // Update the sign
```

```
  await supabase.from('signs').update({
```

```
    ...updates,
```

```
    status: 'edited',
```

```
    edited_by: currentUser.id,
```

```
    updated_at: new Date().toISOString()
```

```
  }).eq('id', signId);
```

```
  // Record the correction for training
```

```
  await supabase.from('training_corrections').insert({
```

```
    tenant_id: currentUser.tenant_id,
```

```
    job_id: jobId,
```

```
    sign_id: signId,
```

```
    room_id: original.room_id,
```

```
    correction_type: 'edited',
```

```

    original_value: original,

    corrected_value: { ...original, ...updates },

    room_name_pattern: extractPattern(original.room_name),

    sign_type: updates.sign_type || original.sign_type,

    created_by: currentUser.id

  });

  // Check for rule override generation

  await checkOverrideThreshold(currentUser.tenant_id, original, updates);
}

```

Training Import Page

The Training Import page allows uploading historical takeoff results (Excel files) that are compared against the rules engine output to generate corrections in bulk.

1. Upload a historical plan PDF + its human takeoff Excel
2. System runs the pipeline on the PDF
3. Diff the pipeline results against the human takeoff
4. Show side-by-side comparison: "Pipeline says X, human says Y"
5. User confirms which corrections are valid
6. Confirmed corrections are bulk-inserted into **training_corrections**
7. Rule overrides are generated from patterns

This is the "training data feedback loop" that improves accuracy over time.

API ROUTES

Auth

POST /api/auth/register

POST /api/auth/login

POST /api/auth/logout

Jobs

GET /api/jobs -- list jobs for tenant

POST /api/jobs -- create new job

GET /api/jobs/{id} -- get job details

PATCH /api/jobs/{id} -- update job

DELETE /api/jobs/{id} -- archive job

POST /api/jobs/{id}/process -- trigger pipeline

POST /api/jobs/{id}/rescan -- re-run pipeline with updated overrides

Files

POST /api/jobs/{id}/files -- upload PDF file(s)

GET /api/jobs/{id}/files -- list files

GET /api/jobs/{id}/sheets -- list identified sheets

Signs

GET /api/jobs/{id}/signs -- list all signs (with filters)

POST /api/jobs/{id}/signs -- manually add a sign

PATCH /api/jobs/{id}/signs/{sid} -- edit a sign

DELETE /api/jobs/{id}/signs/{sid} -- delete a sign (soft delete + training record)

Rooms

GET /api/jobs/{id}/rooms -- list all rooms

GET /api/jobs/{id}/rooms/{rid} -- room details with signs

Exports

GET /api/jobs/{id}/export/xlsx -- download Excel takeoff

GET /api/jobs/{id}/export/pdf -- download marked-up PDF

Training

GET /api/training/corrections -- list corrections for tenant

POST /api/training/import -- bulk import historical takeoff

GET /api/training/overrides -- list rule overrides

PATCH /api/training/overrides/{id} -- toggle/edit override

Plaque Schedule & Occupant Loads

GET /api/jobs/{id}/plaque-schedule

GET /api/jobs/{id}/occupant-loads

GET /api/jobs/{id}/validation

Admin

GET /api/admin/tenants -- super admin only

GET /api/admin/users

POST /api/admin/users/invite

CONFIDENCE SCORING

Each sign gets a confidence score (0-100%):

- **90-100%:** Room label extracted cleanly + rule applied unambiguously + tenant has a confirming override
- **70-89%:** Room label extracted + rule applied, but no confirming override
- **50-69%:** Room label partial match or rule had to make a judgment call (R8 restroom variant, R3 multi-entry)
- **30-49%:** LLM-assisted classification or low-quality text extraction
- **<30%:** Fallback/guess — needs human review

Signs with confidence < 70% get "NEEDS REVIEW" status. Signs with confidence < 50% get "EXCEPTION" status.

KEY METRICS (shown in job header bar)

- **Total Signs Extracted:** count of all signs (including soft-deleted)
- **High Confidence:** count where confidence $\geq 70\%$
- **Needs Review:** count where confidence < 70%
- **AI Processing Cost:** sum of Claude API token costs for this job

IMPORTANT IMPLEMENTATION NOTES

1. **The rules engine is the core product.** It must be deterministic, traceable, and testable. Every sign assigned must have a `rule_ref` tracing back to R1-R15 or a `rule_overrides` entry. No "magic" assignments.
2. **Claude API is used sparingly.** Only for reading 2-3 specific pages visually. The system prompt for each call is ~200 words, not the full methodology. The methodology lives in the Python rules engine code.
3. **Training corrections accumulate per tenant.** Tenant A's corrections don't affect Tenant B's rules. Each tenant builds their own override library over time.
4. **The marked-up PDF must be printable.** Estimators print these and mark them up further by hand. Markers should be small, color-coded, and offset from room labels so the underlying plan text remains readable.
5. **Multi-tenant from day one.** Every query must filter by `tenant_id`. Use Supabase RLS policies on every table. Never serve data cross-tenant.
6. **The floor plan viewer is the hero feature.** It must be smooth (60fps pan/zoom), visually clean, and support hundreds of markers without lag. Use canvas rendering for markers if DOM-based rendering gets slow (>100 markers).
7. **Preserve the existing dark-theme aesthetic.** The current app looks professional and data-dense. Don't lighten it up or add rounded/bubbly UI elements. Keep the Bloomberg-terminal feel.
8. **"Re-scan" button** re-runs the pipeline with the latest rule overrides applied. This is how training improvements get applied to existing jobs.

9. **Every user action on a sign (add/edit/delete) creates a training_corrections record.**
This is non-negotiable — it's how the system learns.
10. **Keep the tab navigation from the current app** — Sign Table, Sheets Analysis, Sign Type Summary, Floor Plans, Sign Pages, Sign Specs, Timeline, Coordinates, AI Scans, Compliance, Plaque Schedule, Occupant Loads. Not all tabs need to work in v1, but the navigation structure should be there.